PySpark Mastery

Topics to Master + Deep-Dive Interview Questions

Azure Data Engineer Edition — Databricks | Synapse | Delta Lake | Structured Streaming

Part 1: Topics to Master | Part 2: Interview Questions (No Answers)

18	UDFs, SparkSQL & Data Formats	Questions	10
19	Real-World Scenario Questions	Questions	12
	TOTAL QUESTIONS		**100**

PART 1 — TOPICS TO MASTER

1. Spark Architecture & Execution Model

1.1 Cluster Architecture

- Driver vs Executor model — roles, memory allocation, JVM heap
- Cluster Manager types: YARN, Kubernetes, Databricks (serverless & classic), Standalone
- SparkContext, SparkSession — entry points and their lifecycle
- DAG (Directed Acyclic Graph) — how Spark plans execution
- Jobs, Stages, Tasks — understanding the execution hierarchy
- Shuffle boundaries — what triggers a new stage

1.2 Memory Model

- Unified Memory Management: execution memory vs storage memory
- spark.memory.fraction and spark.memory.storageFraction — tuning parameters
- On-heap vs off-heap memory — when to use off-heap (Tungsten)
- GC pressure — recognising and mitigating JVM garbage collection issues
- Driver memory vs executor memory — how to configure each

1.3 Execution Model

- Lazy evaluation — transformations are not executed until an action is called
- Catalyst Optimizer — logical plan -> optimised logical plan -> physical plan
- Tungsten engine — code generation, binary processing, cache-aware computation
- Adaptive Query Execution (AQE) — runtime plan changes, skew join handling
- Cost-Based Optimizer (CBO) — table statistics and join reordering
- Reading and interpreting the Spark UI: DAG visualiser, stage details, task metrics

■ *Always open the Spark UI when debugging performance. Stage input/output sizes and shuffle bytes reveal bottlenecks instantly.*

2. RDDs, DataFrames & Datasets

2.1 RDD (Resilient Distributed Dataset)

- RDD fundamentals: partitioning, lineage, fault tolerance
- When to use RDDs: unstructured data, fine-grained control, custom serialisation
- RDD operations: map, flatMap, filter, reduceByKey, groupByKey, sortByKey
- Partitioner: HashPartitioner vs RangePartitioner
- RDD lineage and checkpointing for fault recovery
- Why to avoid RDDs in modern PySpark: no Catalyst optimisation, no Tungsten

2.2 DataFrame API

- Creating DataFrames: from files, from RDDs, from Python lists/dicts, from SQL
- Schema inference vs explicit schema definition with StructType / StructField
- Column operations: col(), lit(), expr(), when/otherwise
- String, date, math, array, map, struct functions from pyspark.sql.functions
- Null handling: isNull(), isNotNull(), fillna(), dropna(), coalesce()
- Working with nested data: explode(), posexplode(), arrays_zip()
- Struct and map type columns: getField(), getItem()

2.3 Dataset API & Type Safety

- Datasets are strongly-typed — available in Scala/Java, not natively in Python
- In PySpark, DataFrames ARE Datasets[Row] — understand this equivalence
- Type safety patterns in PySpark using dataclasses and schema validation

3. Transformations & Actions

3.1 Narrow vs Wide Transformations

- Narrow transformations: map, filter, union — no data movement between partitions
- Wide transformations: groupBy, join, distinct, repartition — trigger shuffle
- How shuffle impacts performance: disk I/O, network transfer, sort
- Minimising wide transformations in pipeline design

3.2 Key Transformations to Master

- select, filter/where, withColumn, withColumnRenamed, drop, alias
- groupBy + agg: sum, count, avg, max, min, collect_list, collect_set, countDistinct
- orderBy / sort with asc/desc — distributed sort implications
- distinct vs dropDuplicates — differences and performance
- pivot: reshaping data from rows to columns
- unionByName vs union — schema alignment
- crossJoin — when and why (and why to avoid it at scale)
- sample(), limit() — for development and testing

3.3 Actions

- collect() — brings all data to driver: dangerous on large datasets
- take(n), first(), head() — safer alternatives for inspection
- count(), show(), printSchema(), describe() — common exploratory actions
- write — triggers execution and materialises results
- foreach(), foreachPartition() — side effects per row / per partition

■ *Avoid collect() on large DataFrames. Use take(20) or write to storage instead — collect() sends all data to the driver and can cause OOM crashes.*

4. Joins & Shuffles

4.1 Join Types

- inner, left, right, full (outer), left_semi, left_anti, cross
- left_semi: equivalent to WHERE EXISTS — only returns left-side matching rows
- left_anti: equivalent to WHERE NOT EXISTS — returns non-matching left rows
- Broadcast join: small table broadcast to all executors — eliminates shuffle
- Sort-merge join: default for large-large joins — requires shuffle + sort
- Shuffle-hash join: used for medium tables — hash bucketing without sort

4.2 Shuffle Management

- spark.sql.shuffle.partitions — default 200, tune to data size (rule: ~128MB/partition)
- Identifying shuffle-heavy stages in Spark UI: shuffle read/write bytes
- Salting technique: adding a random prefix to skewed keys to distribute load
- AQE skew join handling: automatic split of skewed partitions
- Bucketing: pre-partitioned tables that eliminate shuffle on repeated joins
- repartition(n, col) vs repartition(n): hash vs round-robin partitioning
- coalesce(n): reduce partitions without shuffle — only for narrowing

4.3 Data Skew

- How to detect skew: Spark UI task duration variance, partition size checks

- Salting pattern: add random suffix (0-N) to skewed key, join on salted key
- AQE (spark.sql.adaptive.enabled): auto-coalesces small partitions, handles skew
- Skew hints: SKEW_JOIN hint for Spark 3.x

5. Window Functions & Aggregations

5.1 Window Specification

- Window.partitionBy(), Window.orderBy() — defining window scope
- rowsBetween() vs rangeBetween() — frame specification differences
- unboundedPreceding, currentRow, unboundedFollowing — frame boundaries

5.2 Window Function Categories

- Ranking: row_number(), rank(), dense_rank(), ntile(n)
- Analytic: lead(), lag(), first(), last()
- Aggregate: sum(), avg(), min(), max(), count() over a window
- Cumulative distributions: cume_dist(), percent_rank()

5.3 Advanced Aggregations

- cube() and rollup() — multi-dimensional aggregations
- groupingSets() — flexible custom aggregation combinations
- collect_list() vs collect_set() — aggregating into arrays
- approx_count_distinct() — HyperLogLog for large cardinality counts
- percentile_approx() — approximate quantiles at scale
- pivot() with aggregation — reshaping grouped data

■ *Window functions do NOT reduce row count. If you need both the window result and the original rows, window functions are the right tool — not groupBy.*

6. Performance Tuning & Optimisation

6.1 Partitioning Strategy

- Optimal partition size: 128MB–256MB per partition
- repartition vs coalesce: when to use each
- Partition by column for downstream reads: partitionBy() in write
- File compaction: avoid small files problem with OPTIMIZE (Delta) or coalesce before write
- spark.sql.files.maxPartitionBytes — controls Parquet split size on read

6.2 Caching & Persistence

- cache() = persist(StorageLevel.MEMORY_AND_DISK)
- Storage levels: MEMORY_ONLY, MEMORY_AND_DISK, DISK_ONLY, OFF_HEAP
- When to cache: reused DataFrames in iterative loops or multiple action calls
- unpersist() — always release cache when no longer needed
- Caching in Databricks: Delta caching vs Spark caching — know the difference

6.3 Broadcast Variables & Accumulators

- broadcast() — force broadcast of a DataFrame, bypass auto-broadcast threshold
- spark.sql.autoBroadcastJoinThreshold — default 10MB, tune to 50-100MB
- Accumulators — distributed counters/sums for monitoring (not for logic)
- broadcast variables for lookup tables shared across tasks

6.4 Configuration Tuning

- spark.executor.memory, spark.executor.cores, spark.executor.instances
- spark.default.parallelism vs spark.sql.shuffle.partitions

- spark.sql.adaptive.enabled — AQE (always enable in Spark 3.x)
- spark.sql.adaptive.coalescePartitions.enabled — auto-reduce shuffle partitions
- spark.sql.adaptive.skewJoin.enabled — auto handle skewed joins
- spark.serializer: KryoSerializer vs Java serializer (Kryo is faster)

7. Structured Streaming

7.1 Core Concepts

- Micro-batch vs Continuous processing modes
- readStream: sources — Kafka, Event Hub, ADLS Gen2, Delta, socket
- writeStream: sinks — Delta, Kafka, ADLS Gen2, console, memory
- trigger: Trigger.ProcessingTime, Trigger.Once, Trigger.AvailableNow
- outputMode: append, complete, update — when each applies
- checkpointLocation — exactly-once semantics and restart recovery

7.2 Stateful Streaming

- Watermarking: withWatermark() — handling late-arriving data
- Window operations on event time: window(), tumbling vs sliding windows
- Stateful aggregations: groupBy + agg on streaming DataFrames
- Stream-stream joins: event-time joins with watermarks
- Stream-static joins: enriching a stream with a static lookup table
- mapGroupsWithState / flatMapGroupsWithState — custom stateful logic

7.3 Azure Integration

- Azure Event Hub as Kafka-compatible source in PySpark
- Connection string configuration for Event Hub in Spark
- Writing streaming output to Delta Lake tables
- Streaming Delta table reads — using readStream on a Delta table
- Monitoring streaming queries: query.status, query.lastProgress, query.awaitTermination()

■ *Always set a checkpointLocation for production streaming jobs — without it, you lose all state on restart and cannot guarantee exactly-once processing.*

8. Delta Lake & Databricks

8.1 Delta Lake Core

- Transaction log (_delta_log): JSON commit files, checkpoint files
- ACID transactions on data lake storage
- DeltaTable API: DeltaTable.forPath(), forName()
- MERGE INTO (upsert): matched, notMatched, notMatchedBySource clauses
- UPDATE and DELETE on Delta tables
- Time travel: VERSION AS OF, TIMESTAMP AS OF in PySpark
- Schema enforcement (default) vs schema evolution (mergeSchema option)
- OPTIMIZE and Z-ORDER: file compaction and data skipping
- VACUUM: removing old files, default 7-day retention
- Change Data Feed (CDF): table_changes() for incremental reads

8.2 Databricks-Specific

- dbutils.fs, dbutils.secrets, dbutils.widgets, dbutils.notebook
- Delta caching vs Spark caching — Delta caches on SSD, Spark in memory
- Auto Loader (cloudFiles): incremental file ingestion from ADLS Gen2
- Unity Catalog: three-level namespace (catalog.schema.table)
- Databricks Workflows: job clusters, task dependencies, retries

- Photon engine: C++ vectorised query engine — when it accelerates workloads

9. UDFs, Pandas UDFs & Spark SQL

9.1 Python UDFs

- Registering UDFs: udf() decorator and spark.udf.register()
- Return type specification — always declare the return type explicitly
- Null handling in UDFs — UDFs do not handle nulls by default
- Performance cost: row-by-row Python serialisation, no Catalyst integration
- When to use UDFs: complex business logic not expressible in built-in functions

9.2 Pandas UDFs (Vectorised UDFs)

- Series to Series UDF: element-wise vectorised transformations
- Series to Scalar UDF: aggregation returning a scalar
- Iterator of Series UDF: batch processing with state initialisation
- pandas_udf decorator with PandasUDFType or type hints (Spark 3.x)
- 10-100x faster than Python UDFs due to Apache Arrow serialisation
- Use cases: ML scoring, NLP, complex maths, library calls (scipy, numpy)

9.3 Spark SQL

- spark.sql() — run SQL against temp views and Delta/Hive tables
- createOrReplaceTempView() vs createOrReplaceGlobalTempView() — scope differences
- SQL and DataFrame API interoperability — mix freely within a session
- SQL hints: BROADCAST, COALESCE, REPARTITION, SKEW_JOIN
- EXPLAIN EXTENDED — reading SQL query plans from Python

■ *Replace Python UDFs with Pandas UDFs or built-in functions wherever possible. Pandas UDFs use Apache Arrow and are typically 10-100x faster than row-level Python UDFs.*

10. Data Formats & I/O

- Parquet: columnar, splittable, schema-embedded — default for data lakes
- Delta: Parquet + transaction log — preferred for Azure DE workloads
- Avro: row-based, schema registry integration, good for streaming/Kafka
- JSON: flexible but slow — avoid for large datasets, use schema inference carefully
- CSV: lowest common denominator — always specify schema, handle bad records
- ORC: columnar, used in Hive-based ecosystems
- Read options: schema, header, inferSchema, multiline, badRecordsPath, mode
- Write options: mode (overwrite/append/ignore/errorIfExists), partitionBy, compression
- Reading from ADLS Gen2: abfss:// protocol with Managed Identity
- Predicate pushdown and column pruning — how Parquet/Delta optimise reads
- spark.read.option('mergeSchema', True) — handling schema evolution across files

PART 2 — DEEP-DIVE INTERVIEW QUESTIONS (No Answers)

Section 11 — Architecture & Core Concepts

Q1. Explain the Spark Driver and Executor model. What happens when the driver fails? What happens when an executor fails?

Q2. What is a DAG in Spark? How does Spark build and optimise the DAG before execution? What is the role of the Catalyst optimizer?

Q3. What is lazy evaluation in PySpark? Give a concrete example where it improves performance and one where it causes confusion.

Focus: Execution model

Q4. Explain the difference between a Job, Stage, and Task in Spark. What triggers a new stage boundary?

Q5. What is the Tungsten execution engine? How does it improve on the original Spark execution model?

Q6. Explain Adaptive Query Execution (AQE). What three key optimisations does it perform at runtime?

Q7. What is the difference between spark.default.parallelism and spark.sql.shuffle.partitions? When does each apply?

Q8. You have a Spark job that runs fine in development (100GB data) but crashes in production (10TB). What are the likely causes and how do you fix it?

Focus: Scaling issues

Q9. Explain the unified memory model in Spark. What is the difference between execution memory and storage memory?

Q10. What is a shuffle in Spark? Describe exactly what happens on disk and over the network during a shuffle operation.

Q11. Explain the Cost-Based Optimizer (CBO) in Spark. How do you enable it and what statistics does it use?

Q12. What is speculative execution in Spark? When would you enable it and what are its risks?

Q13. You open the Spark UI and see one task taking 10 minutes while all others finish in 10 seconds. What is this called and how do you fix it?

Focus: Data skew

Q14. What is the difference between client mode and cluster mode when submitting a Spark job? How does this affect driver placement?

Section 12 — DataFrames, Schemas & APIs

Q15. What is the difference between an RDD and a DataFrame in PySpark? When would you still use an RDD in modern PySpark?

Q16. Explain schema inference vs explicit schema definition. What are the performance and correctness risks of schema inference in production?

Q17. What is the difference between col(), lit(), and expr() in PySpark? Give an example where expr() is the best choice.

Q18. Explain the difference between select() and withColumn(). When can withColumn() cause performance issues when called many times?

Focus: Performance trap

Q19. How does PySpark handle null values? Explain the behaviour of null in filter(), join(), groupBy(), and agg().

Q20. You have a DataFrame with a column containing arrays of structs. How would you flatten it into one row per array element with all struct fields as columns?

Focus: Nested data

Q21. What is the difference between unionByName() and union()? What happens when schemas don't match?

Q22. How do you read a directory of Parquet files where different files have different schemas (schema evolution)? What option do you set?

Q23. What is the difference between orderBy() and sortWithinPartitions()? When would you use each?

Q24. Explain the difference between distinct() and dropDuplicates(). When would you use dropDuplicates with a subset of columns?

Q25. How do you efficiently add multiple new columns to a DataFrame? What is the anti-pattern and what is the correct approach?

Q26. What is a broadcast variable in PySpark? How is it different from a broadcast join? Give an example of each.

Section 13 — Transformations, Joins & Shuffles

Q27. Explain the difference between a broadcast join and a sort-merge join. How does Spark decide which to use?

Q28. What is a left_semi join and left_anti join? Write a PySpark example for each and explain when you'd use them over a filter.

Q29. You are joining a 500GB fact table with a 10MB dimension table in Spark. The join is extremely slow. What is the problem and how do you fix it?

Focus: Broadcast join

Q30. Explain data skew in joins. Describe the salting technique in detail — how do you implement it in PySpark code?

Q31. What is bucketing in Spark? How does it eliminate shuffle for repeated joins and how do you create a bucketed table?

Q32. You need to join two 100GB DataFrames on a key that has 80% of rows with the same value. What strategies do you use?

Focus: Severe skew

Q33. What is the difference between repartition(n) and repartition(n, col)? When does partitioning by column benefit downstream operations?

Q34. Explain the difference between repartition() and coalesce(). When should you NOT use coalesce()?

Q35. What happens under the hood when you call groupBy().agg() in PySpark? Describe the map and reduce phases.

Q36. You have a DataFrame of 1 billion rows and need to deduplicate on a composite key. What is the most efficient approach?

Focus: Large-scale dedup

Q37. What is reduceByKey vs groupByKey in RDD API? Why is reduceByKey preferred and what is the performance difference?

Q38. How does the SKEW_JOIN hint work in Spark 3.x? Compare it to the manual salting approach.

Q39. You notice spark.sql.shuffle.partitions is set to 200 but your data is 2TB. What is the impact and what should you set it to?

Q40. Explain partition pruning in Spark. How do you write a DataFrame to storage so that downstream reads benefit from partition pruning?

Section 14 — Window Functions & Aggregations

Q41. Explain the difference between rowsBetween() and rangeBetween() in a Spark window specification. Give an example where they produce different results.

Q42. What is the difference between row_number(), rank(), and dense_rank()? Write a PySpark query showing all three on the same dataset.

Q43. Using PySpark window functions, write code to calculate a 7-day rolling average of sales per product, handling missing days correctly.

Q44. How would you identify the first and last event per session for each user using PySpark window functions?

Focus: Session analytics

Q45. Explain the difference between cube() and rollup(). Give an Azure DE scenario where you would use each for a sales reporting pipeline.

Q46. What is the performance implication of using collect_list() in a groupBy aggregation on a large DataFrame? How do you mitigate it?

Q47. Write a PySpark query to assign customers into deciles (10 equal groups) based on their total spend, within each region.

Focus: ntile + window

Q48. How do you compute the percentage contribution of each product to its category total using window functions — without a self-join?

Q49. What is approx_count_distinct() and when would you use it over countDistinct()? What algorithm does it use?

Q50. Using PySpark, write a gaps-and-islands solution to find consecutive active days per user from a daily_logins table.

Section 15 — Performance Tuning

Q51. A PySpark job that used to run in 20 minutes now takes 3 hours. The code hasn't changed but data volume tripled. Walk through your diagnostic process.

Focus: Performance regression

Q52. Explain the difference between Spark caching and Delta caching in Databricks. When would you use each?

Q53. What are the storage levels in PySpark persist()? When would you use MEMORY_AND_DISK over MEMORY_ONLY?

Q54. You have a Spark job with 10,000 shuffle partitions and tasks are all completing in milliseconds. What is the problem and how do you fix it?

Focus: Too many partitions

Q55. Explain predicate pushdown and column pruning. How do they work with Parquet and Delta Lake to reduce I/O?

Q56. What is GC overhead in Spark and how do you diagnose it? What configuration changes and code changes can reduce GC pressure?

Q57. When should you use off-heap memory (spark.memory.offHeap.enabled)? What types of workloads benefit most?

Q58. Explain the auto-broadcast join threshold. How do you force a broadcast join when the table is just over the threshold?

Q59. What is the small files problem in Spark? How does it manifest in read performance and what are the solutions?

Q60. You have a PySpark job that calls withColumn() 150 times in a loop. It runs extremely slowly. Why? How do you fix it?

Focus: withColumn anti-pattern

Q61. What is dynamic partition overwrite in Spark? How does it differ from static partition overwrite mode?

Q62. How do you tune executor memory and cores for a memory-intensive PySpark job on Databricks? What is the memory overhead factor?

Q63. Explain Kryo serialisation vs Java serialisation in Spark. How do you enable it and for what types of jobs does it matter most?

Q64. What is the Photon engine in Databricks? What types of operations does it accelerate and which does it not yet support?

Section 16 — Structured Streaming

Q65. Explain the difference between micro-batch processing and continuous processing in Spark Structured Streaming. When would you use each?

Q66. What are the three output modes in Structured Streaming — append, complete, update? When can each be used and what are their restrictions?

Q67. What is a checkpoint in Structured Streaming? What exactly is stored in the checkpoint directory and why is it critical for production jobs?

Q68. Explain watermarking in Structured Streaming. How do you configure it and what happens to late-arriving events that exceed the watermark?

Q69. What is the difference between Trigger.ProcessingTime, Trigger.Once, and Trigger.AvailableNow? When would you use Trigger.AvailableNow over a batch job?

Q70. How do you implement a stream-static join in PySpark Structured Streaming? What are the refresh strategies for the static side?

Q71. Describe how you would consume Azure Event Hub data in PySpark Structured Streaming. What are the key configuration parameters?

Focus: Azure Event Hub

Q72. A streaming job writes to a Delta table but after a restart it reprocesses all data from the beginning. What went wrong?

Focus: Checkpoint failure

Q73. What is flatMapGroupsWithState? Give a real-world use case — such as session window tracking — where it is the right tool.

Q74. How do you perform a windowed aggregation by event time (not processing time) in Structured Streaming? What is the difference?

Q75. You have a streaming job reading from ADLS Gen2 using Auto Loader. Explain how Auto Loader tracks which files have been processed.

Focus: Auto Loader

Q76. How do you monitor a production Structured Streaming job on Databricks? What metrics do you track and how do you set up alerting?

Section 17 — Delta Lake & Databricks

Q77. Explain how the Delta Lake transaction log works. What is stored in each commit JSON file and what is a checkpoint file?

Q78. Write a PySpark MERGE INTO statement to implement an SCD Type 2 pattern on a customer dimension table. Walk through the logic.

Focus: SCD Type 2

Q79. What is Z-Ordering in Delta Lake? How does it improve query performance and what are its limitations?

Q80. Explain the difference between OPTIMIZE and VACUUM in Delta Lake. What happens if you run VACUUM with a retention period shorter than the default?

Q81. What is Change Data Feed (CDF) in Delta Lake? How do you enable it and how do you read incremental changes using table_changes()?

Q82. How does schema enforcement work in Delta Lake? How do you enable schema evolution for a MERGE or streaming write?

Q83. Explain Delta Lake time travel. Write PySpark code to read a table at a specific version and to restore a table to a previous version.

Q84. What is the small files problem in Delta Lake and how does OPTIMIZE solve it? How do you automate OPTIMIZE in Databricks?

Q85. You are concurrently writing to a Delta table from two Databricks jobs. One job fails with a ConcurrentModificationException. Explain why and how to resolve it.

Focus: Concurrency

Q86. What is Auto Loader in Databricks? Compare it to a standard Spark readStream on a directory. What are its advantages?

Q87. How do you implement a Medallion Architecture (Bronze/Silver/Gold) using Delta Lake in Databricks? Describe the responsibility of each layer.

Q88. What is Unity Catalog in Databricks? How does it change data governance compared to Hive Metastore?

Section 18 — UDFs, Spark SQL & Data Formats

Q89. Explain the performance cost of a Python UDF in PySpark. Why is it slower than a built-in function, and how does a Pandas UDF (vectorised UDF) improve on it?

Q90. Write a Pandas UDF to apply a scikit-learn ML model for batch inference on a PySpark DataFrame. What Series-to-Series UDF type would you use?

Focus: ML inference

Q91. What are the null-handling differences between a Python UDF and a built-in Spark function? How do you make a UDF null-safe?

Q92. What is the difference between createOrReplaceTempView() and createOrReplaceGlobalTempView()? When would you use the global version?

Q93. How do you use EXPLAIN EXTENDED in PySpark? What does each plan (parsed, analysed, optimised, physical) tell you?

Q94. Compare Parquet, Delta, Avro, and JSON for data lake storage. For each format, name one scenario where it is the best choice for an Azure DE.

Q95. What is predicate pushdown for Parquet? How does adding a filter on a non-partitioned column still reduce data scanned?

Q96. How do you handle bad records (malformed rows) when reading CSV or JSON in PySpark? What are the mode options and the badRecordsPath?

Q97. What is the mergeSchema option when reading Parquet? When would you set it to True and what are the risks?

Q98. You need to read a JSON column containing nested arrays and structs in a Delta table and flatten it to a relational schema. Describe your PySpark approach.

Focus: Nested JSON

Section 19 — Real-World Scenario Questions

Q99. Scenario: You have a daily batch job that reads 2TB of Parquet from ADLS Gen2, joins with a 500MB lookup table, and writes the result to a Delta table. The job takes 4 hours. Walk through every optimisation you would apply.

Focus: End-to-end optimisation

Q100. Scenario: A PySpark streaming job consuming from Azure Event Hub is falling behind — the lag is growing continuously. The cluster has plenty of resources. Diagnose the root cause and describe your remediation steps.

Focus: Streaming performance

Q101. Scenario: Your Delta table has 2 million small files (avg 50KB each) after 6 months of streaming writes. Reads are now taking 10x longer than expected. Describe your remediation plan without causing downtime.

Focus: Small files at scale

Q102. Scenario: Two data engineers are running concurrent MERGE operations on the same Delta table from separate jobs. You are seeing intermittent job failures. Explain what is happening and redesign the pipeline to be concurrency-safe.

Focus: Concurrent writes

Q103. Scenario: You are building a near-real-time sales dashboard. Events arrive via Event Hub, need to be enriched with a product dimension (updated hourly), aggregated by hour and region, and exposed via a Delta Gold table. Design the full PySpark pipeline.

Focus: Streaming + Delta architecture

Q104. Scenario: A data science team needs to run batch ML inference using a custom Python library (not available as Spark built-ins) on 500 million rows. Standard UDFs are too slow. Design the most performant PySpark solution.

Focus: Pandas UDF at scale

Q105. Scenario: Your Spark job shuffles 500GB of data during a groupBy aggregation. The job has 200 shuffle partitions and completes in 3 hours. How do you tune it to run in under 30 minutes?

Focus: Shuffle tuning

Q106. Scenario: You are implementing a GDPR right-to-erasure requirement. A user's data must be deleted from Bronze, Silver, and Gold Delta tables, plus all previous versions. How do you implement this in PySpark + Delta Lake?

Focus: GDPR erasure on Delta

Q107. Scenario: You need to process 10 years of historical transaction data (50TB) as a one-time backfill into Delta Lake using Databricks. Design the ingestion strategy for maximum throughput and cost efficiency.

Focus: Large-scale backfill

Q108. Scenario: A PySpark job reads from a Delta table partitioned by date. Even though your filter is WHERE date = '2024-01-01', the Spark UI shows it is reading all partitions. Explain the possible causes and how you fix each.

Focus: Partition pruning failure

Q109. Scenario: You are joining a 1TB transaction table with a 900GB customer table. Both are too large to broadcast. The join key (customer_id) is severely skewed — 30% of rows share the top 10 customer IDs. Design the complete PySpark solution.

Focus: Large skewed join

Q110. Scenario: You need to implement an incremental pipeline that detects new and changed records in a source Delta table and propagates them to a Gold aggregation table — without reprocessing unchanged data. Design the solution using Delta CDF and PySpark.

Focus: Incremental with CDF
